

Optimizing Conway's Game of Life

Ethan Tran

Group 421 | Topic 13 | ISYE-6644 | Georgia Institute of Technology

Abstract

Conway's Game of Life (often abbreviated as "GoL" and "Life") was proposed by John Conway in 1970, to experiment with the concept of self-replicating machines. The game itself is a cellular automaton that takes place on an infinite, boundless grid, consisting of cells that are either living or dead, and progression of time occurs in discrete "ticks". The rules that govern the game universe are simple and few:

1. Living cells with less than two living neighbors die
2. Living cells with two or three living neighbors continue to live
3. Living cells with over three living neighbors die
4. Dead cells with three living neighbors become living

The game accepts minimal user input; the only involvement done on a human "player's" part is done before the simulation starts, setting the initial seed of the universe.

Background

Overview

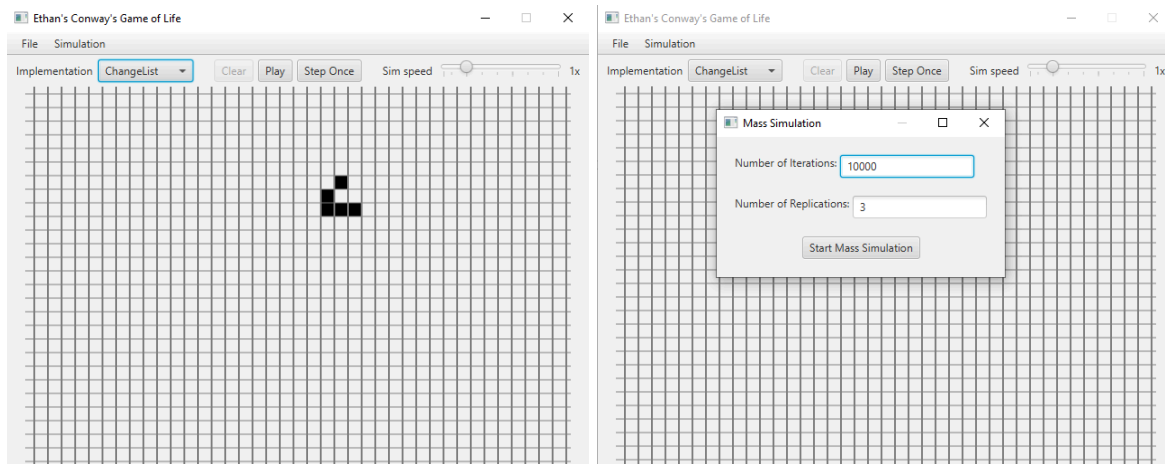
The goal of this project is to implement the GoL. I've long been fascinated with the topic ever since I started my studies in computer science, and this course was an opportune moment to finally throw myself into the practice. One large problem with the GoL, however, is that despite the simplicity of its rules, an efficient implementation can be deceptively hard to devise. In order to make observations on a long lived simulation, performance is paramount. However, the need to satisfy all of the specifications of the game--from its cell interactions to its infinite grid--are computationally demanding. This demand for efficiency provides an excellent opportunity for the application of advanced computer science concepts. The 50-year lifetime of the GoL has bred a good number of optimizations to choose from.

Method

General Concepts

In order to test the effectiveness of different approaches to software implementations of the GoL, I utilized my self-flagellating passion for programming to create a platform for running Life simulations. The device is written in Java, with a convenient graphical user interface I spent an irresponsible amount of time creating. Due to the nature of some of some featured implementations, the tool features a finitely-bounded grid of configurable sizes of powers of 2, however some implementations (namely Hashlife) have the potential to scale with an infinite bound. The feature of this tool most relevant for research purposes is the mass simulation

window, which performs large-scale GoL simulations and documents iteration execution times in a convenient csv spreadsheet. For the sake of configurability, iteration and replication counts for each implementation featured can be defined by the user.



I learned JavaFX just for this

As with the functionality provided in the testing tool, computational time will be the primary metric for performance measurements; however memory management is also a valid point of concern I had not been able to measure. In the following tests, I will use this simulator as a test bench for recording execution times of different implementations (as well as test replications), calculating averages across replications per iteration, and comparing these averages against other implementations. These measurements will be saved to a spreadsheet, where they can be processed into meaningful conclusions.

Implementation 1: The Naive Approach

As a basis for comparison (and a general warmup/introduction to the cellular automata ethos), I implemented an incredibly simple and logically naive method for calculating Life evolutions: A two-dimensional boolean array for storing cell data, and an engine that would, per evolution, iterate over every single cell in the matrix to calculate cell interactions. It likely needs no further explanation on its inefficiency; for a grid of length and width n , the system would iterate n^2 times and, per cell, run checks on its 8 neighbors to apply the Life rules. Increasing world sizes quadratically increases evolution calculation times, to a point in which a 8192x8192 grid would take almost 3 seconds to compute a single tick. I did not take the time to simulate en masse using this method, as the iteration counts I saw fit to use (10,000) would take far too much time to perform. Thankfully, inferences can easily be made on a hypothetical total simulation time, as the time complexity of such a simple system should remain constant.

Implementation 2: Recording Changed Cells

There are a good number of observations on the behavior of the Game of Live one can leverage to optimize the performance of a simulation. One such idea is the fact that change in a Life system occurs quite locally; dead cells are guaranteed to remain dead if isolated from any

entropic source (e.g. a live cell). That is to say: only the cells adjacent to cells that have recently changed are susceptible to change. Utilizing this knowledge, one could drastically improve upon the design of a naive implementation by recording the cells that have changed--or are adjacent to a changed cell--in a previous tick, and applying the Rules only to them. Central to this approach is a structure to hold a *unique* set of two-dimensional coordinates (the consequences of failure in this regard are dire), and I have used the Java *HashSet* structure to accomplish this. The performance returns are immense; runtime is no longer defined by the size of the universe bounds, only by the number of cells that have changed over the past iteration.

Implementation 3: Hashlite

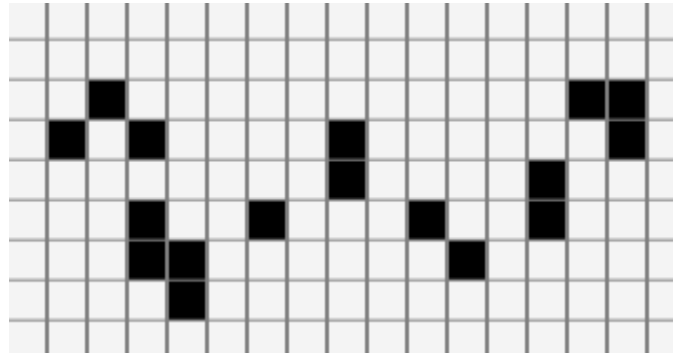
Ah, hashlife. Please forgive this moment of admiration for the algorithm. I have had my eye on this implementation before I had chosen to undertake this project assignment, as it incorporates an immense amount of algorithmic concepts in its design. This algorithm, devised and implemented by others far brighter than I, incorporates the usage of Quadtrees to represent the universe (Note: many of the articles I consulted on hashlife were quite lacking on concrete code sampling, at least for Java. I ended up having to figure out the fine details of the algorithm myself, which was just as enlightening as it was harrowing. I studied it for a month before committing any real code to the file). The nodes in these trees are memoized and stored in a repository to be reused wherever their representative patterns reappear in the universe, and nodes are, themselves, composed of smaller nodes. The products of evolution are also memoized to speed up calculations for identical patterns. For this approach, the titular Hashmap is used to index quadtree nodes based on their child nodes, and a separate Hashmap caches nodes to their respective evolution results. It is in the technique of solution caching that powerful computational optimizations can be made: The Rules are not calculated at runtime, but are applied through the recognition of already-known patterns. This approach comes at a somewhat dramatic startup cost, though; my implementation of Hashlife prepopulates the node cache with all 65536 possibilities of a 4x4 binary grid, though this startup overhead proves to be worth the effort in the long run, as will be addressed in the next section regarding performance measurements. Alas, I was not able to fully implement the principles of Hashlife, as I did not have the time to address the time-compressive capabilities of caching multi-tick evolutions. Thankfully, the algorithm is still a great optimization without these strategies (and may have added additional complexity to the long-simulation test system, anyway). I have opted to dub this incomplete version of the algorithm "Hashlite", for distinction.

Results

Performance Comparison

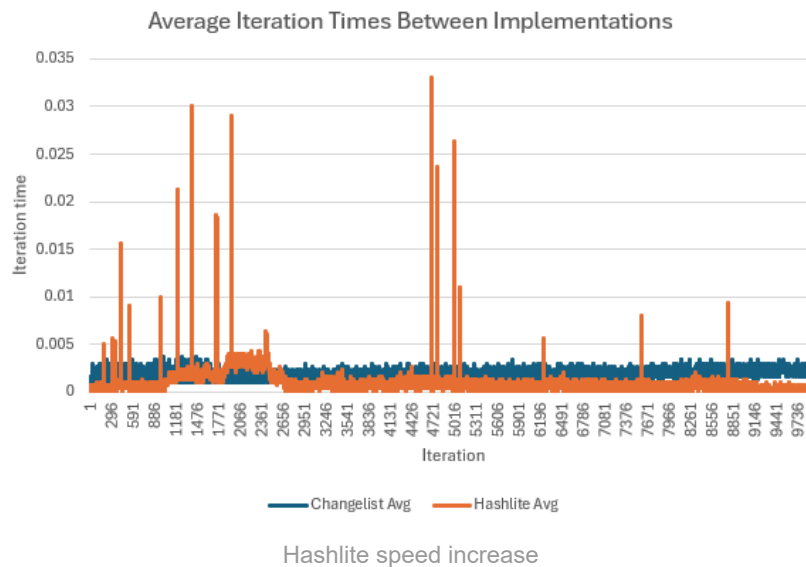
As mentioned before, I will not bother with Naive approach metrics, as they are far too inefficient to simulate and measure. For the Changelist and Hashmap implementations, I ran a simulation on a fairly explosive Life pattern (known as the "time bomb") for 10,000 iterations, and calculated the time elapsed between each evolution. This would provide an opportunity for the implementations to slow down to cater to the complexities of the rapidly evolving patterns. The

testbench saves these observations to a csv file, in which I use my barely passable skills in Excel to perform analyses on the runtime measurements.



The Time Bomb

The results of these tests were... expected and unsurprising, yet nonetheless fascinating to see in practice: Hashlite, despite its fluctuations in iteration time, amortizes to an overall *faster* runtime compared to the Changelist approach. It has a spike in runtime at startup as a result of its complex initialization, and later has a slow start at the first fourth of the simulation, but speeds up as the simulation continues by virtue of its evolution memoization. This nets it a significant performance increase over the Changelist approach, clocking an average total of 9.38 seconds compared to the Changelist's 16.75 second runtime for a difference of 7.37 seconds.



Conclusion

Takeaways

The journey to produce this code was, frankly, a painful one. I had tread onto a good number of subjects I was not familiar with: between setting up a Maven project, learning to use JavaFX for the GUI, and making an application that ran well, I had spent approximately 2 weeks of

dedicated work unrelated to the GoL implementation. However, I take pride in the quality of the software and the amount of science applied in this project. And there is still much to be done: the program still lacks a means of measuring the memory footprint of an approach, and Hashlife still remains to be fully implemented. Thankfully, this tool provides an extensible platform for testing and measuring GoL simulations, and is engineered to easily allow for the addition of features and Life approaches.

References

<https://conwaylife.com/book/>

<https://conwaylife.com>

https://conwaylife.com/wiki/Conway%27s_Game_of_Life

<https://www.dev-mind.blog/hashlife/>

<https://johnhw.github.io/hashlife/index.md.html>

<https://www.drdobbs.com/jvm/an-algorithm-for-compressing-space-and-t/184406478>